

SDLC

Requirement analysis

Planning

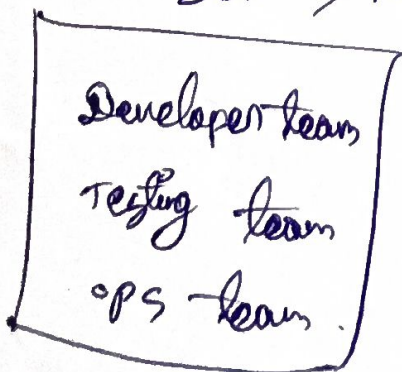
Design

Development

Deployment & maintenance

Linux $\Rightarrow$ sshwindow $\Rightarrow$ RDPAMI $\Rightarrow$ pre-install templateInstance store $\Rightarrow$ non-persistent

g: ATM

install^{OS} $\Rightarrow$ Bootable deviceEBS $\Rightarrow$ it is a bootableAgileMaster $\rightarrow$ monitor

standup meet (or) Scrum meet

phic $\Rightarrow$ multiple separate
 $\hookrightarrow$ Complete application build and deploy.

Sprint: 1) short cycles.

↳ short time full focus.

Devops:

Plan

tools:

JIRA → ethic, sprint, use pannalam

Code

tools:

SCM → source code manager

tools:

→ Git Hub, Git, Git lab.

Build:

maven → build tool and dependency tool.

React

for python → Pytorch

file

• .js

• .for

• .jar

Test:

selenium → front end web browser automate selenium
Postman.

release (CI) Integration

Jenkins, GitHub Actions.

deploy

Docker, Kubernetes

operate:

Kubernetes, Docker Swarm

monitor → cloud watch, prometheus, Grafana → visualize data.

DevOps

It is a software development methodology which improves the collaboration b/w developers & operations team using various automation tools. These automation tools are implemented using various stages which are a part of the DevOps Life Cycle.

CI/CD Process

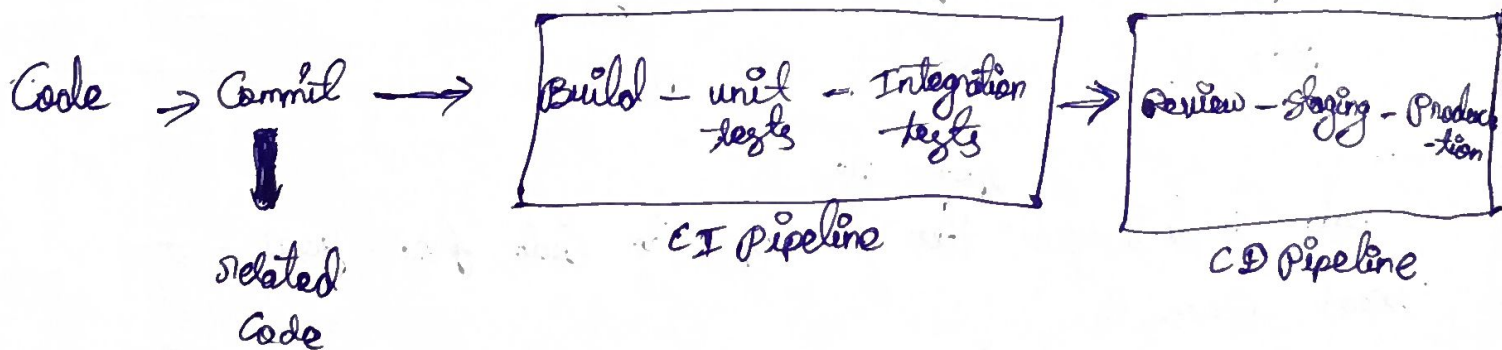
Commit change $\rightarrow$ Build $\rightarrow$ Test $\rightarrow$ Deliver $\rightarrow$ Deploy

Automated CI/CD

Staging $\Rightarrow$ Test server.

Production $\Rightarrow$ Production server.

Pipeline



GitHub $\Rightarrow$ object storage.

object storage $\Rightarrow$ cloud storage.

block storage $\Rightarrow$ pen drive, hard drive.

Gray box test $\Rightarrow$ Both combined white & black box test. They are know the internal & external code.

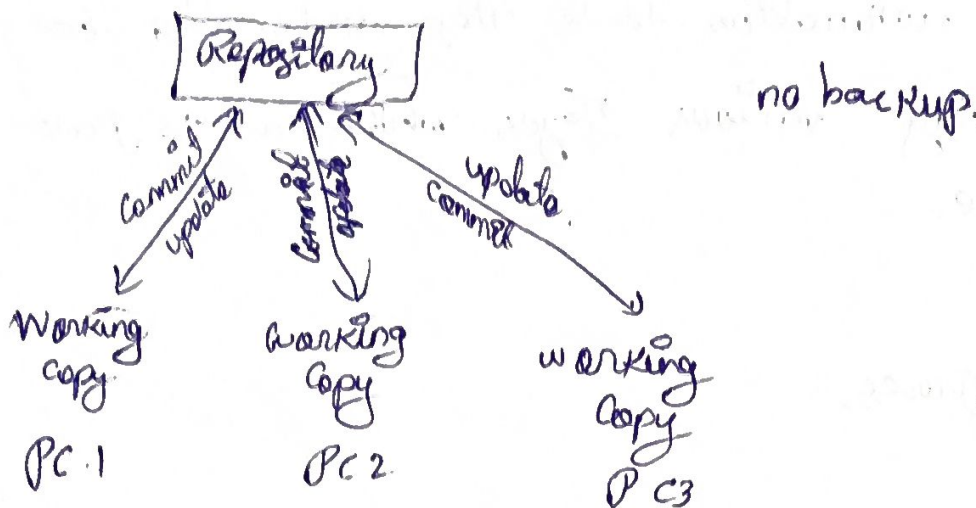
white box test $\Rightarrow$ Test the source code

Black box test $\Rightarrow$ test the functionality & non-functionality
it is not visible.

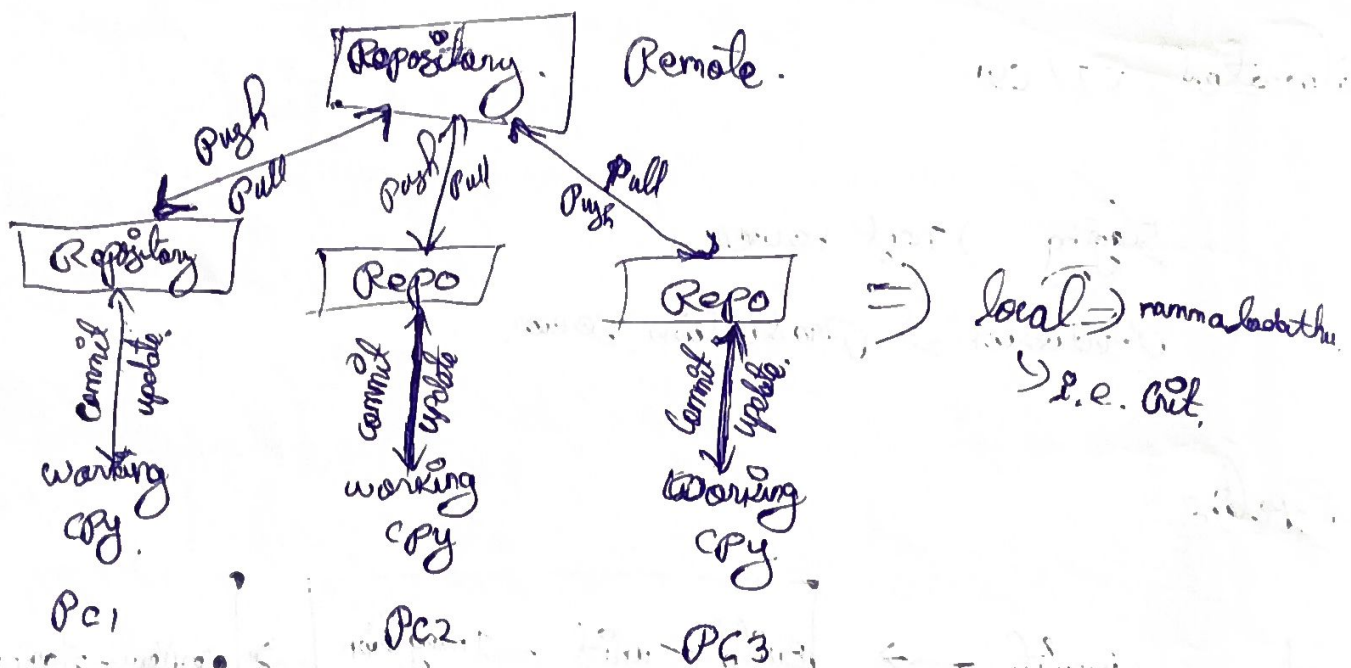
Source code management.

merge conflicts $\Rightarrow$ b/w branches.

Centralized version control system:



Distributed version control system:



Code is destroyed ^{from} ~~to~~ then backup the Code from your local Repository.

Suppose code destroyed from local repository then backup the code from Remote.

Push $\Rightarrow$ Local to Remote.

Pull $\Rightarrow$ Remote to Local.

Pull Request $\Rightarrow$ work complete Parvita says Pull

shell $\Rightarrow$ linux support pannam.

local repo

$\rightarrow$ mkdir sic-devops

$\rightarrow$ git init

$\rightarrow$ cd sic-devops

$\rightarrow$ git init

$\rightarrow$ vi text.txt

$\rightarrow$ cat text.txt $\Rightarrow$

$\rightarrow$ ~~remove~~ the inside statement.
change

$\rightarrow$ after exit without saving.

linux

vi $\Rightarrow$ one of the
(or) text editing
Vim tool

save and exit

ESC

:wq

Enter.

exit without saving.

:q!

$\rightarrow$ git config --global user.email "abc@gmail.com".

git add. $\Rightarrow$ all files added.

git add text.txt $\Rightarrow$ specific file add.

$\rightarrow$ change working directory to staging directory.

LF $\Rightarrow$ new line

CRLF $\Rightarrow$ carries return line feed

LF to CRLF $\Rightarrow$ file closed.

This is working directory to staging directory.

git commit -m "demo commit"

$\rightarrow$ reference

git status

git log $\Rightarrow$ commit panna files ellam. show pannam.

local - to remote connect.

git ignore -la oruola file-a ignore pannanum
athoda name kuduskanum.

~~git pr~~

git remote add origin "url"
↳ github repo url.

git push

Generate the key.

ssh-keygen

Enter
Enter

protocol secure panra

cat /c/users/gi

GitHub-ku oruola ingala-la setting → SSH keys
athoda key page pannanum.

ls -al ~/.ssh
clip < ~/.ssh/id_ed25519.pub → Public key

suppose
this cmd not working
use this

cat ~/.ssh/id_ed25519.pub

then copy the key page in GitHub.

branch change cmd:

git branch -M main

git push origin main

Suppose any error type this cmd.

git remote add origin "github repo url."